

Ludo Project- SRS Document

Group: Panda



GROUP MEMBERS:
STAV ELKRIEF- 314777988
HILA KASA- 208906164
LANA ADMUZ- 21338480
SHIRA MARDER- 323085233

Contents

1.Introduction 3
1.1 Purpose: 3
1.2 Scope: 3
1.3 Definitions, acronyms, and abbreviations 3
1.5 Overview: 3
2. Overall Description 3
2.1 Product Perspective: 3
2.2 Product Functions: 3
2.3 User Characteristics: 4
2.4 Constraints: 4
2.5 Assumptions and Dependencies: 4
2.6 Apportioning of Requirements: 4
3. Specific Requirements Organized by Feature 4
3.1 External Interface Requirements 4
- 3.1.1 User Interfaces: 4
- 3.1.2 Hardware Interfaces: 4
- 3.1.3 Software Interfaces: 4
- 3.1.4 Communications Interfaces: 4
- 3.2 System Features 3.2.1 Feature 1: Board Configuration and Setup 5
- 3.2.2 Feature 2: Piece Movement 6
- 3.2.3 Feature 3: Surprise Square Events 8
- 3.2.4 Feature 4: Question Handling 10
- 3.2.5 Feature 5: Question Bank Administration 11
- 3.2.6 Feature 6: Game Data Persistence 12
- 3.2.7 Feature 7: Game History Viewing 13
- 3.2.8 Feature 8: Turn and Dice Management 14
- 3.2.9 Feature 9: Game Completion 16

3.3 Performance Requirements 17
- 3.3.1 PR 1 - Board Configuration and Setup: 17
- 3.3.2 PR 2 - Piece Movement: 17
- 3.3.3 PR 3 - Surprise Square Events: 17
- 3.3.4 PR 4 - Question Handling: 18
- 3.3.5 PR 5 - Question Bank Administration: 18
- 3.3.6 PR 6 - Game Data Persistence: 18

- 3.3.7 PR 7 - Game History Viewing: 18
- 3.3.8 PR 8 - Turn and Dice Management: 18
- 3.3.9 PR 9 - Game Completion:....18

3.4 Design Constraints 18

3.5 Software system attributes 18

3.6 Other requirements –..... 19

1.Introduction

1.1 Purpose: This SRS defines the requirements for an extended Ludo game, enabling course staff to evaluate its functionality, architecture, and design patterns.

1.2 Scope: The system is a standalone desktop Ludo application for 2–4 players, featuring three difficulty levels (Easy, Medium, Hard), trivia questions, and Steps, Boost, and UltraBoost dice. It includes secure administration of at least 30 course-related multiple-choice questions and persistent CSV storage of completed-game history.

1.3 Definitions, acronyms, and abbreviations

- **Double Block:** A state where two or more pieces belonging to the same player stand in the same square, preventing opposing pieces from passing through or landing on it.
- **Safe Square:** A square where a piece cannot be eaten/captured.
- **Question Square:** A square that triggers a multiple-choice question matching the square's color-coded difficulty (Red=Hard, Yellow=Medium, Green=Easy).
- **Surprise Square:** A safe square that triggers a random game event based on fixed statistical probabilities.
- **Boost/ UltraBoost Dice:** Custom dice yielding specific multipliers to modify piece movement.
- **MVC:** Model-View-Controller architecture.

1.4 References

- IEEE Recommended Practice for Software Requirements Specifications (IEEE Std 830-1998).
- "Ludo project" instructions and game rules document, Department of Information Systems, University of Haifa.

1.5 Overview: Section 2 describes the system and its constraints; Section 3 specifies its requirements.

2. Overall Description

2.1 Product Perspective: The system is a standalone Java desktop application using a CSV file for game history and a different file for question storage. Development is managed through GitHub.

2.2 Product Functions: The major functions the software will perform include:

- **Game Configuration:** Setting up the game board to Easy, Medium, or Hard configurations, which scale the board size and the number of special squares.
- **Turn Execution:** Calculating piece movement based on standard dice rolls and executing specialized movement based on Boost or UltraBoost dice.
- **Square Interactions:** Managing interactions when pieces land on standard, safe, question, or surprise squares.

- **Trivia Integration:** Pulling questions from a bank of course-related topics (e.g., Agile, MVC, Design Patterns, QA) and applying game penalties or rewards based on the player's answer.
- **Administration:** Providing a secure management interface to add, edit, or delete questions in the Question bank.
- **Game Completion and History:** Ranking players, displaying final results, and storing and viewing completed-game history.

2.3 User Characteristics: The system is designed for two distinct classes of users:

- **Players:** General users who interact with the graphical user interface to roll dice, move pieces, and answer multiple-choice questions. They require no special technical expertise.
- **Administrator:** An authorized user who manages the question bank.

2.4 Constraints: The system is subject to Java, MVC architecture, Design Patterns, a fixed CSV history schema, and code reuse across difficulty levels. These constraints are detailed in Section 3.4.

2.5 Assumptions and Dependencies: The application requires a Java-compatible desktop environment. Development and collaboration use GitHub.

2.6 Apportioning of Requirements: is irrelevant for this system.

- **Justification:** According to IEEE 830, this section identifies requirements deferred to future versions of the system. As this academic assignment requires a single, fully functional final submission demonstrating all required capabilities (including MVC architecture and edge case handling), no future commercial versions are planned, making this subsection irrelevant.

3. Specific Requirements Organized by Feature

Template Choice Justification: Template A.5 was selected because the Ludo system is primarily organized around distinct functionalities rather than user classes or operating modes. Grouping requirements by feature allows related behaviors and rules to be specified together, improving clarity, traceability, and verification while minimizing duplication.

3.1 External Interface Requirements

❖ **3.1.1 User Interfaces:**

- The system shall provide a GUI for gameplay, dice rolls, question handling, game history, and question bank administration.

❖ **3.1.2 Hardware Interfaces:** The system shall support standard desktop input/output devices.

❖ **3.1.3 Software Interfaces:** The system shall use CSV for game history.

❖ **3.1.4 Communications Interfaces:**

- No network communication is required.

❖ **3.2 System Features**

3.2.1 Feature 1: Board Configuration and Setup

- **3.2.1.1 Introduction:** This feature allows players to configure and start a new Ludo game. The system creates the appropriate board for the selected difficulty level, initializes players and their pieces, and places the required squares according to the game rules.
- **3.2.1.2 Stimulus/Response Sequence:**

1. A player selects the option to start a new game.
2. The system requests the number of players and the game difficulty level.
3. The players enter their names or nicknames and select distinct colors.
4. The system configures the board according to the selected difficulty, including the required paths and square types.
5. The system randomly positions Safe, Question, and Surprise squares while preserving their required quantities and placement constraints.
6. The system places four pieces for each player in the corresponding castle and displays the initialized board, ready for the first turn.

- **3.2.1.3 Associated Functional Requirements:**
  - **3.2.1.3.1 FR 1.1- Number of Players:** The system shall allow the configuration of a new game with 2 to 4 players.
    Priority: 3. **Justification:** The supported number of players is a fundamental requirement of the game.
  - **3.2.1.3.2 FR 1.2- Player Identification:** The system shall allow each player to enter a name and select a unique color.
    Priority: 3. **Justification:** Player identification and unique colors are required to distinguish players and their pieces.
  - **3.2.1.3.3 FR 1.3- Difficulty Selection:** Players shall select Easy, Medium, or Hard difficulty before starting; the selection remains fixed.
    Priority: 3. **Justification:** The difficulty level determines the required board configuration and remains fixed during gameplay.
  - **3.2.1.3.4 FR 1.4- Board Configuration by Difficulty:** The system shall generate the game board according to the selected difficulty level:
    Easy: A 52-square common path, 4 Safe (Star-marked) squares, 4 Arrow squares, and 4 Starting squares.
    Medium: The 52-square common path and an additional 64-square path, 8 Safe (Star-marked) squares, 8 Arrow squares, 6 Question squares (2 per question difficulty level), 2 Surprise squares, and 4 Starting squares.
    Hard: The 52-square common path, the additional 64-square path, and a further 72-square path, 12 Safe (Star marked) squares, 12 Arrow squares, 9 Question squares (3 per question difficulty level), 3 Surprise squares, and 4 Starting squares.

Question squares shall be color-coded according to their difficulty: Green for Easy, Yellow for Medium, and Red for Hard.
Priority: 3. **Justification:** The system must support all three board configurations and their required square quantities as defined in the project instructions.

- **3.2.1.3.5 FR 1.5 - Fixed Square Placement:** The system shall place Starting and color-specific Arrow squares at predefined board positions.
  Priority: 3. **Justification:** Fixed square positions and color associations are essential for maintaining the required board structure and routes.
- **3.2.1.3.6 FR 1.6 - Random Placement of Special Squares:** The system shall randomly place Safe, Question, and Surprise squares at distinct eligible positions on shared paths, preserving the required quantities for the selected difficulty level. Randomly placed special squares shall not overlap fixed Starting or Arrow squares. Adjacent special squares shall be permitted.
  Priority: 3. **Justification:** Random placement must preserve required quantities and placement constraints.
- **3.2.1.3.7 FR 1.7- Castles and Home Paths:** Each player shall have a castle and a six-square home path leading to the finish.
  Priority: 3. **Justification:** Castles and home paths are essential components of the Ludo board.
- **3.2.1.3.8 FR 1.8 - Initial Piece Placement:** Each player shall start with four pieces of their selected color in their castle.
  Priority: 3. **Justification:** Each player must begin the game with four pieces in the designated castle, as required by the game rules.

❖ **3.2.2 Feature 2: Piece Movement**

- **3.2.2.1 Introduction:** This feature manages piece movement, captures, Double Blocks, backward movement, and finishing rules.
- **3.2.2.2 Stimulus/Response Sequence:**

1. The system receives the movement steps calculated by the Turn and Dice Management feature.
2. The player selects a piece to move or, if permitted, to exit the castle.
3. The system validates and executes the move, applying Arrow and Double Block rules.
4. The system resolves captures and triggers applicable special-square events.
5. The system updates piece positions and the board state.

- **3.2.2.3 Associated Functional Requirements:**
  - **3.2.2.3.1 FR 2.1 - Entering the Board/Castle Exit:** The system shall allow a player to move a piece out of the castle only when the Steps Die shows six. Castle exit shall be prohibited if an opponent's Double Block occupies the player's Starting square. A single

opponent piece on the Starting square shall not prevent castle exit and shall not be captured.
Priority: 3. **Justification:** Castle exit must follow the dice rules and respect Starting square protection and Double Block restrictions.

- **3.2.2.3.2 FR 2.2 - Castle Exit with Zero Multiplier:** A roll of six permits castle exit even when the movement multiplier is zero.
  Priority: 2. **Justification:** This requirement handles a specific exception to the regular movement calculation.
- **3.2.2.3.3 FR 2.3 - Forward Piece Movement:** The system shall move pieces clockwise by the calculated steps.
  Priority: 3. **Justification:** Forward movement along the defined board paths is essential to gameplay.
- **3.2.2.3.4 FR 2.4 - Arrow Square Movement:** In Medium and Hard modes, pieces shall begin their movement on the outer path. When a piece reaches its color-matched Arrow square, the system shall transition it to the next inner path using any remaining movement steps. The piece shall not be required to complete a full lap before transitioning. Arrow squares of other colors shall be treated as regular squares.
  Priority: 3. **Justification:** Arrow squares determine the required transitions between board paths, allowing pieces to advance to inner paths without completing a full lap.
- **3.2.2.3.5 FR 2.5 - Capturing Opponent Pieces:** Landing on a single unprotected opponent piece returns it to its castle, requiring another six to re-enter.
  Priority: 3. **Justification:** Capturing opponent pieces is a fundamental Ludo mechanic.
- **3.2.2.3.6 FR 2.6 - Protected Squares:** Pieces on Safe, Starting, Surprise, and Home squares cannot be captured.
  Priority: 3. **Justification:** Protected squares must provide the capture protection defined in the game rules.
- **3.2.2.3.7 FR 2.7 - Double Block Formation:** The system shall recognize two or more same-player pieces on any square, including Starting squares, as a Double Block. The player's own pieces may pass through or land on it.
  Priority: 3. **Justification:** Double Block formation and interaction are required movement mechanics.
- **3.2.2.3.8 FR 2.8 - Double Block Movement Restrictions:** The system shall prevent opponents from passing through or landing on a Double Block, forward or backward. If blocked, the player shall move another piece capable of completing the full move, if available; otherwise, the blocked piece shall stop immediately before the block, if reachable. If an opponent's piece is already on a protected square when a Double Block is formed on that square, the system shall allow that piece to leave the square through a legal move.

Priority: 3. **Justification:** Double Blocks must restrict opposing pieces according to the game rules and the clarified partial-movement conditions, without trapping an opponent's piece already occupying a protected square.

- **3.2.2.3.9 FR 2.9 - Backward Piece Movement:** When a Question or Surprise event requires backward movement, the system shall move the affected piece backward under applicable restrictions, allowing captures on unprotected squares. If the move would pass the piece's own Starting square (#), it shall stop there instead of returning to the castle.
  Priority: 3. **Justification:** Backward movement and its interactions with captures and obstacles are required by the Question and Surprise Square mechanics.
- **3.2.2.3.10 FR 2.10 - Home Path and Final Finish:** The system shall guide pieces along their personal home paths to the central finish. Pieces already on a home path shall move using only the Steps Die (FR 8.4). Finishing requires exact steps, including Question or Surprise effects, overshooting is prohibited except under FR 3.1.
  Priority: 3. **Justification:** Home-path movement and exact-step finishing must follow game rules.
- **3.2.2.3.11 FR 2.11- Movement Outcomes and Square Interactions:** After movement, the system shall evaluate the landing square, resolve applicable captures before Question events, and trigger Question or Surprise events when required.
  Priority: 3. **Justification:** Movement outcomes must be processed consistently so that captures and special- square interactions follow the game rules.

❖ **3.2.3 Feature 3: Surprise Square Events**

- **3.2.3.1 Introduction/ Purpose of Feature:** This feature manages probability-based events triggered by landing on Surprise squares, which protect pieces from capture and may affect movement, turns, or dice rolls.
- **3.2.3.2 Stimulus/Response Sequence:**

1. A piece lands on a protected Surprise square, triggering an event.
2. The system randomly selects one of six events according to predefined probabilities.
3. The system executes the event, applying relevant movement and turn rules, including any required selections or dice rolls.
4. The system updates the game state and processes any resulting square interactions according to the applicable game rules.

- **3.2.3.3 Associated Functional Requirements:**
  - **3.2.3.3.1 FR 3.1 - Direct Finish Event:** When selected, the system shall move the triggering piece directly to its final finish square, regardless of distance or intervening Double Blocks.

Priority: 2. **Justification:** This event is a required exception to regular movement and finishing rules.

- **3.2.3.3.2 FR 3.2 - Random Surprise Event Selection:** Upon landing on a Surprise square, the system shall randomly select exactly one event with the following probabilities:
  - Direct movement to the final finish square- 10%.
  - Move an opponent's piece backward by half the Steps Die result- 10%.
  - Return the triggering piece to its castle- 10%.
  - Move the triggering piece three steps backward- 30%.
  - Skip the player's next turn- 30%.
  - Grant a special Steps Die and UltraBoost Die roll- 10%.

Priority: 3. **Justification:** All six Surprise events and their predefined probabilities are mandatory game mechanics.

- **3.2.3.3.3 FR 3.3 - Opponent Piece Movement Event:** When the Opponent Piece Movement event is selected, the system shall roll the Steps Die and let the triggering player choose an opponent and one of their eligible pieces. The system shall move that piece backward by half the result, rounded up, under Piece Movement rules. If the opponent has no eligible piece, another opponent may be selected. If none has an eligible piece, the event shall have no movement effect.
  Priority: 3. **Justification:** Defines opponent selection, movement calculation, and unavailable-move handling.
- **3.2.3.3.4 FR 3.4 - Return to Castle Event:** When the Return to Castle event is selected, the system shall return the piece that triggered the Surprise event to its corresponding castle.
  Priority: 3. **Justification:** Returning a piece to its castle is one of the required Surprise events.
- **3.2.3.3.5 FR 3.5 - Three-Step Backward Movement Event:** When the Backward Movement event is selected, the system shall move the piece that triggered the Surprise event three steps backward, subject to the movement restrictions defined in the Piece Movement feature.
  Priority: 3. **Justification:** The backward movement event must follow the required movement and blocking rules.
- **3.2.3.3.6 FR 3.6 - Skip Next Turn Event:** When the Skip Next Turn event is selected, the system shall cancel the player's next turn. If the player is entitled to an immediate additional roll due to rolling six, that additional roll shall be canceled instead. The event shall not cause an additional skipped turn.

Priority: 3. **Justification:** Skipping a turn is a required Surprise event, and its interaction with additional turns must follow the clarified game rules.

- **3.2.3.3.7 FR 3.7- Special UltraBoost Roll Event:** When the Special UltraBoost event is selected, the system shall grant the player a special additional roll using the Steps Die and UltraBoost Die, according to the rules defined in FR 8.11.
  Priority: 3. **Justification:** The special UltraBoost roll is a required Surprise event and must be consistent with the turn and dice management rules.

❖ **3.2.4 Feature 4: Question Handling**

- **3.2.4.1 Introduction:** This feature manages course-related multiple-choice questions triggered by Question squares, selecting questions by square difficulty, evaluating answers, and applying movement rewards or penalties.
- **3.2.4.2 Stimulus/ Response Sequence:**
  1. A piece lands on a Question square.
  2. The system resolves any applicable capture before presenting a question.
  3. The system identifies the square's difficulty by color and randomly selects a matching question.
  4. The system displays the question and four answer options to all players.
  5. The triggering player selects an answer.
  6. The system evaluates the answer against the stored correct answer.
  7. The system moves the triggering piece forward for a correct answer or backward for an incorrect answer, according to question difficulty.
  8. The system applies movement restrictions, resolves resulting captures, and triggers applicable special-square effects.
  9. The system updates the board and returns control to Turn and Dice Management.
- **3.2.4.3 Associated Functional Requirements:**
  - **3.2.4.3.1 FR 4.1- Question Square Activation:** The system shall trigger a question when a piece lands on a Question square through a valid movement, including movement caused by another special-square effect.
    Priority: 3. **Justification:** Question activation must apply consistently whenever a valid movement causes a piece to land on a Question square.
  - **3.2.4.3.2 FR 4.2 - Random Question Selection:** The system shall randomly select a question from the bank category matching the square's color: Green (Easy), Yellow (Medium), or Red (Hard), regardless of overall game difficulty.
    Priority: 3. **Justification:** Question selection must match the square's difficulty.
  - **3.2.4.3.3 FR 4.3 - Question Display and Answer Selection:** The system shall display the question with exactly four answers to all players, allowing only the triggering player to respond.

Priority: 3. **Justification:** The system must enforce the required question format and answer permissions.

- **3.2.4.3.4 FR 4.4 - Answer Evaluation:** The system shall compare the player's answer with the stored correct answer to determine its correctness.
  Priority: 3. **Justification:** Accurate evaluation is required to apply the appropriate reward or penalty.
- **3.2.4.3.5 FR 4.5 - Correct Answer Reward:** For a correct answer, the system shall move the triggering piece forward according to question difficulty: Easy- 1 step; Medium- 2 steps; Hard- 3 steps.
  Priority: 3. **Justification:** Correct answers must grant the specified movement rewards.
- **3.2.4.3.6 FR 4.6 - Incorrect Answer Penalty:** For an incorrect answer, the system shall move the triggering piece backward according to question difficulty: Easy - 1 step, Medium - 2 steps, Hard - 3 steps.
  Priority: 3. **Justification:** Incorrect answers must produce the movement penalties explicitly defined in the game rules.
- **3.2.4.3.7 FR 4.7 - Question-Induced Movement Restrictions:** The system shall apply Feature 2 movement rules to question-induced moves, including Double Blocks, backward limits, and exact-step finishing.
  Priority: 3. **Justification:** Question-induced movement must comply with movement and finishing rules.
- **3.2.4.3.8 FR 4.8 - Capture Before Question Presentation:** When a piece lands on an unprotected Question square occupied by a single opponent piece, the system shall capture that piece before presenting the question. The capture shall remain valid regardless of the answer or subsequent movement.
  Priority: 3. **Justification:** Required processing order must preserve captures regardless of question outcomes.
- **3.2.4.3.9 FR 4.9 - Subsequent Square Interactions:** After question-induced movement, the system shall process the resulting landing-square interactions according to FR 2.11, including any applicable captures and special square effects.
  Priority: 3. **Justification:** Question induced movement must follow the same landing-square interaction rules as other valid movements.

❖ 3.2.5 Feature 5: Question Bank Administration

- **3.2.5.1 Introduction:** This feature allows an authorized administrator to manage a question bank containing at least 30 course-related multiple-choice questions across Easy, Medium, and Hard levels.
- **3.2.5.2 Stimulus/Response Sequence**

1. The user selects Question Bank Administration and enters the requested credentials.

2. The system authenticates the user and denies access if authentication fails.
3. The system displays existing questions and the add, edit, and delete options.
4. The administrator selects an action and provides the required information.
5. The system validates the request, rejecting invalid changes with an appropriate message.
6. Upon a valid change, the system updates and displays the question bank.

- **3.2.5.3 Associated Functional Requirements:**
  - **3.2.5.3.1 FR 5.1 - Administrator Authentication:** The system shall restrict access to question bank management functions to authenticated administrators.
    Priority: 3 . **Justification:** Preventing unauthorized access is an explicit project security requirement.
  - **3.2.5.3.2 FR 5.2 - View Questions:** The system shall allow authenticated administrators to view stored questions, their difficulty levels, four answer options, and the designated correct answer.
  - **Priority:** 3. **Justification:** Viewing question details is necessary for administrators to review and manage the question bank.
  - **3.2.5.3.3 FR 5.3 - Add Questions:** The system shall allow authenticated administrators to add questions with text, four options, one correct answer, and a difficulty level.
    Priority: 3. **Justification:** Adding questions is a required management function.
  - **3.2.5.3.4 FR 5.4 - Edit Questions:** The system shall allow authenticated administrators to modify question text, options, correct answers, and difficulty levels.
    Priority: 3. **Justification:** Editing questions is required to maintain the question bank.
  - **3.2.5.3.5 FR 5.5 - Delete Questions:** The system shall allow an authenticated administrator to delete a question only if the remaining question bank satisfies the minimum-content requirement defined in FR 5.6. Otherwise, the system shall reject the deletion and display an explanatory warning.
    Priority: 3. **Justification:** Question deletion must preserve the required minimum content of the question bank.
  - **3.2.5.3.6 FR 5.6 - Question Bank Content:** The system shall maintain at least 30 course-related multiple-choice questions across Easy, Medium, and Hard levels.
    Priority: 3. **Justification:** The minimum question quantity, format, topics, and difficulty levels are specified in the project requirements.
  - **3.2.5.3.7 FR 5.7 - Question Bank Persistence:** The system shall store and retrieve the question bank in a separate file, preserving authorized changes across sessions.
    Priority: 3. **Justification:** Authorized changes must persist across sessions.

❖ **3.2.6 Feature 6: Game Data Persistence**

- **3.2.6.1 Introduction:** This feature stores completed-game history in the required CSV format, preserving records across sessions.

- **3.2.6.2 Stimulus/Response Sequence:**

1. The game ends, providing final rankings and duration.
2. The system collects the required data and assigns a unique gameID.
3. The system saves the record to the CSV file, preserving existing history.
4. The record becomes available for history viewing. If saving fails, the system notifies the user.

- **3.2.6.3 Associated Functional Requirements:**
  - **3.2.6.3.1 FR 6.1 - Completed Game Storage:** The system shall save completed-game results to a persistent CSV file.
    Priority: 3. **Justification:** Persistent storage of completed game history is a mandatory project requirement.
  - **3.2.6.3.2 FR 6.2 - Mandatory Results Storage:** The system shall generate a unique gameID and store each completed game's results using the exact, unalterable CSV schema: gameID, difficulty, timestamp(DD/MM/YYYY; HH:MM:SS), 1st_place_name, 2nd_place_name, 3rd_place_name, 4rd_place_name, game_duration(HH:MM:SS).
    Priority: 3. **Justification:** Fulfills the strict course constraint to persistently save all required game data using the provided CSV format.
  - **3.2.6.3.3 FR 6.3 - Game Time Information:** The system shall store game timestamps and durations in the formats defined in the CSV template.
    Priority: 3. **Justification**: Both time-related fields are mandatory components of the history record and must follow the required formats.
  - **3.2.6.3.4 FR 6.4 - Persistent History:** The system shall retain previously saved game records when a new completed game is added.
    Priority: 3. **Justification:** Game history must remain available across multiple game sessions.
  - **3.2.6.3.5 FR 6.5 - History Retrieval:** The system shall allow stored game history records to be retrieved from the CSV file for display.
    Priority: 3. **Justification:** Stored history must be accessible through the required game history functionality.

❖ **3.2.7 Feature 7: Game History Viewing**

- **3.2.7.1 Introduction:** This feature displays completed-game history, including difficulty, timestamp, player rankings, and duration.
- **3.2.7.2 Stimulus/Response Sequence:**

1. The user selects Game History from the main menu.
2. The system retrieves completed-game records from the CSV file.
3. If records exist, the system displays them, including player rankings and game

duration.

4. Otherwise, the system displays a message indicating that no game history is available.
5. The user may return to the main menu.

- **3.2.7.3 Associated Functional Requirements:**
  - **3.2.7.3.1 FR 7.1 - Access Game History:** The system shall provide a Game History option in the main menu.
    Priority: 3. **Justification:** Game history viewing is a required function.
  - **3.2.7.3.2 FR 7.2 - Display Completed Games:** The system shall display the retrieved game history records, including each game's difficulty, timestamp, player rankings, and duration.
    Priority: 3. **Justification:** Users must be able to access previously saved game results.
  - **3.2.7.3.3 FR 7.3 - Empty History Handling:** If no completed game records are available, the system shall display an appropriate message.
    Priority: 2. **Justification:** Ensures clarity when no game history exists.
  - **3.2.7.3.4 FR 7.4 - Return to Main Menu:** The system shall allow users to return from the Game History screen to the main menu.
    Priority: 2. **Justification:** Users must be able to navigate back to the application's main functions.

❖ **3.2.8 Feature 8: Turn and Dice Management**

- **3.2.8.1 Introduction:** This feature manages difficulty-based dice rolls, movement calculations, extra rolls, consecutive-six restrictions, and turn transitions.
- **3.2.8.2 Stimulus/Response Sequence:**
  1. The system identifies the active player and displays the appropriate dice.
  2. The player rolls the dice, and the system displays the results.
  3. The system checks FR 8.6. If a third consecutive regular six occurs, it cancels the roll and ends the turn.
  4. For a valid roll, the system calculates the movement steps according to the applicable dice rules.
  5. The system determines legal movement options based on the board state and dice results.
  6. If a legal move exists, the player selects a piece, and the system executes the move (Feature 2). Otherwise, the system applies FR 8.9.
  7. If another roll is permitted, the system continues the player's turn.
  8. Otherwise, the system passes the turn to the next eligible player.
- **3.2.8.3 Associated Functional Requirements:**
  - **3.2.8.3.1 FR 8.1 - Difficulty-Based Dice Selection:** The system shall use the Steps Die in Easy mode, Steps and Boost Dice in Medium mode, and Steps and UltraBoost Dice in

Hard mode.
Priority: 3. **Justification:** Each difficulty level requires a specific dice configuration.

- **3.2.8.3.2 FR 8.2 - Steps Die:** The system shall generate integers from 1 to 6 with equal probability for each possible result.
  Priority: 3. **Justification:** The Steps Die is required at all difficulty levels.
- **3.2.8.3.3 FR 8.3 - Boost and UltraBoost Dice:** The system shall support Boost Die (1–3) and UltraBoost Die (1–6), with equally probable results for each die.
  Priority: 3 . **Justification:** These dice are required for movement calculations in Medium and Hard modes.
- **3.2.8.3.4 FR 8.4 - Movement Calculation:** The system shall calculate shared-path movement as S in Easy mode and $S \times (M - 1)$ in Medium/Hard modes or special UltraBoost rolls (FR 8.11), where S is the Steps Die result, and M is the applicable Boost/UltraBoost result. Pieces already on home paths shall move S steps only.
  Priority: 3. **Justification:** Movement calculations must follow the required dice and home-path rules.
- **3.2.8.3.5 FR 8.5 - Additional Turn After Rolling Six**: A Steps Die result of six shall grant an additional roll, subject to FR 8.6 and FR 3.6.
  Priority: 3. **Justification:** An additional roll after six is a mandatory game rule.
- **3.2.8.3.6 FR 8.6 - Three Consecutive Sixes:** If the Steps Die shows six on three consecutive regular rolls in one turn, the system shall cancel the third roll and its movement and pass the turn, regardless of the Boost or UltraBoost result. Special Surprise rolls (FR 8.11) shall neither count toward nor reset this sequence.
  Priority: 3. **Justification:** The game rules explicitly restrict three consecutive rolls of six.
- **3.2.8.3.7 FR 8.7 - Single-Piece Movement Per Roll:** The system shall allow only one eligible piece to move per valid roll; movement steps shall not be split among pieces.
  Priority: 3. **Justification:** Each roll must apply to one piece.
- **3.2.8.3.8 FR 8.8 - Mandatory Legal Movement:** If at least one legal movement option is available, the system shall require the player to perform a legal move rather than voluntarily skipping the turn.
  Priority: 3. **Justification:** The game rules require players to perform a move when a legal option exists.
- **3.2.8.3.9 FR 8.9 - No Legal Movement:** If no legal movement option is available, the system shall prevent movement and grant an additional roll on a Steps Die result of six, subject to FR 8.6; otherwise, it shall pass the turn.
  Priority: 3. **Justification:** The system must correctly handle turns in which movement is not possible.

- **3.2.8.3.10 FR 8.10 - Zero Movement Multiplier:** A Boost or UltraBoost result of one shall produce zero shared-path movement. Home-path movement, castle exit, and additional-roll eligibility shall remain governed by FR 8.4, FR 2.2, and FR 8.5, respectively. Zero-step movement shall not reactivate Question or Surprise squares. **Priority:** 3. **Justification:** Zero multiplier handling must preserve the applicable exceptions without triggering square effects when no movement occurs.
- **3.2.8.3.11 FR 8.11 - Special UltraBoost Turn:** When a Surprise event grants a special additional turn, the system shall allow a Steps Die and UltraBoost Die roll regardless of difficulty. The player may move any eligible piece or exit the castle on a Steps Die result of six. Such a six shall grant another roll, subject to applicable six-roll rules. The special roll shall neither count toward nor reset the regular consecutive-six sequence. **Priority:** 3. **Justification:** This mandatory Surprise event must follow the applicable dice and turn rules.

❖ **3.2.9 Feature 9: Game Completion**

- **3.2.9.1 Introduction:** This feature manages player completion, rankings, and game termination. A player finishes when all four pieces reach the finish area; the game ends when only one player remains unfinished.
- **3.2.9.2 Stimulus/Response Sequence:**

  1. A piece reaches the finish area, and the system updates its status.
  2. The system checks whether all four of the player's pieces have finished.
  3. If so, the system marks the player as completed, assigns a ranking, and excludes them from further turns.
  4. The system checks the number of unfinished players. If more than one remains, gameplay continues.
  5. If only one player remains unfinished, the system assigns that player the last rank and ends the game.
  6. The system displays the final rankings, difficulty, and game duration.
  7. The system sends the results to the Game Data Persistence feature.
- **3.2.9.3 Associated Functional Requirements:**
  - **3.2.9.3.1 FR 9.1 - Individual Player Completion:** The system shall mark a player as completed when all four of their pieces reach the designated finish area. **Priority:** 3. **Justification:** Reaching the finish area with all four pieces is the required objective for each player.
  - **3.2.9.3.2 FR 9.2 - Player Ranking:** The system shall rank players in the order they complete all four pieces. **Priority:** 3. **Justification:** Rankings must reflect completion order.
  - **3.2.9.3.3 FR 9.3 - Completed Player Status:** The system shall preserve completed players' rankings and exclude them from subsequent turns.

Priority: 3. **Justification:** A player who has completed the game should no longer participate in active gameplay.

- **3.2.9.3.4 FR 9.4 - Overall Game Completion:** The system shall end the game when all but one participating player have finished all four pieces.
  Priority: 3. **Justification:** The game must end when the required completion condition is met.
- **3.2.9.3.5 FR 9.5 - Last Player Ranking:** Upon game completion, the system shall automatically assign the remaining unfinished player the last rank.
  Priority: 3. **Justification:** Final rankings must include all players.
- **3.2.9.3.6 FR 9.6 - Final Results Display:** The system shall display final rankings, selected difficulty, and total game duration.
  Priority: 3. **Justification:** Displaying the final game information is part of the required system flow.
- **3.2.9.3.7 FR 9.7 - Completed Game Data Transfer:** Upon game completion, the system shall transfer final rankings, difficulty, and duration to the Game Data Persistence feature for storage in the required history format.
  Priority: 3. **Justification:** Completed game information must be available for persistent storage and future history viewing.

3.3 Performance Requirements

This section defines the system's performance requirements, organized by feature. It specifies the supported capacities and processing workloads, with references to Section 3.2 for the corresponding functional requirements.

- **3.3.1 PR 1 - Board Configuration and Setup:** The system shall support initializing a game with 2-4 players and up to 16 pieces in each difficulty level. The initialization workload shall include the complete board configuration, special-square placement, and placement validation specified in Section 3.2.1, including the maximum board configuration required for Hard difficulty.
- **3.3.2 PR 2 - Piece Movement:** At the maximum game capacity defined in PR 1, the system shall support evaluating the movement options for the active player's pieces and processing the selected move. The workload shall include all applicable movement rules and interactions specified in Section 3.2.2, including interactions with other pieces and transitions between board paths.
- **3.3.3 PR 3 - Surprise Square Events:** The system shall support processing every Surprise event specified in Section 3.2.3 at the maximum game capacity. Event processing shall include random selection using the specified probabilities, any required player or piece selection, and the resulting movement or turn management processing defined in Sections 3.2.2 and 3.2.8.

- **3.3.4 PR 4 - Question Handling:** The system shall support selecting and displaying questions from a bank containing at least 30 questions across three difficulty levels. The processing workload shall include answer evaluation and the resulting movement processing, as specified in Sections 3.2.4 and 3.2.2.
- **3.3.5 PR 5 - Question Bank Administration:** The system shall support loading and managing a question bank containing at least 30 questions, each with four answer options, one correct answer, and a difficulty level. The supported workload shall include the administration and persistence operations specified in Section 3.2.5. The minimum of 30 questions shall not be treated as a maximum storage capacity.
- **3.3.6 PR 6 - Game Data Persistence:** The system shall support storing a completed-game record containing results for up to four players in an existing history file. Record contents, file format, and preservation of earlier records are specified in Section 3.2.6.
- **3.3.7 PR 7 - Game History Viewing:** The system shall support retrieving and displaying stored game history records with the fields specified in Section 3.2.7. The workload for this feature depends on the number of stored records.
- **3.3.8 PR 8 - Turn and Dice Management:** The system shall support turn and dice processing for up to four participating players across all difficulty levels. The workload shall include the calculations, turn transitions, and regular or special additional rolls specified in Section 3.2.8, together with the movement processing referenced in Section 3.2.2.
- **3.3.9 PR 9 - Game Completion:** The system shall support completion processing and final ranking for up to four players. The workload shall include completion checks, ranking updates, final results display, and transfer of completed game data for storage, as specified in Sections 3.2.9 and 3.2.6.

3.4 Design Constraints

This section defines the mandatory technical and architectural constraints governing the system's design and implementation.

1. **Programming Language:** The application shall be implemented in Java.
2. **Software Architecture:** The system shall follow the Model-View-Controller (MVC) architecture and incorporate appropriate Design Patterns.
3. **Data Format:** Game history shall use the mandatory CSV schema provided by the course staff, preserving its field names, order, and formats.
4. **Code Reuse:** Common gameplay logic shall be reused across the three difficulty levels: Easy, Medium, and Hard.

3.5 Software system attributes

This section describes the software quality attributes applicable to the Ludo system, based on the mandatory project requirements.

- **Reliability:** The system shall maintain a consistent game state by enforcing the specified movement, turn management, and game completion rules, including handling invalid moves and edge cases, as defined in Section 3.2.
- **Availability:** Not applicable as a separate requirement.
  Justification: The system is a standalone desktop application, and the project specifications do not define continuous availability, uptime, or recovery requirements. Persistent storage and retrieval of completed-game history are addressed in Features 6 and 7.
- **Security:** The system shall restrict access to question bank administration and protected question data to authenticated administrators, preventing unauthorized users from accessing or modifying restricted information, as specified in Feature 5.
- **Maintainability:** The system shall organize shared gameplay rules in reusable components so that modifications to common rules do not require separate changes for each difficulty level. The mandatory architectural constraints are defined in Section 3.4.
- **Portability:** The system shall be implemented in Java, and the submitted executable shall be verified to run correctly on more than one compatible desktop computer, as required by the project instructors.
- **Usability:** The system shall provide a graphical user interface that displays the game board, current player, game time, and applicable dice during gameplay. The interface shall support access to game setup, game history, and restricted question administration, as specified in the project requirements.

3.6 Other requirements – The system shall be delivered as a compiled executable that is verified to run on more than one compatible desktop computer without requiring an IDE. The source code shall include clear documentation describing the responsibility of each class.

Team Work Division:

Hila Kasa - Overall System Description, Question Bank Administration, Turn Management, and Software Quality Attributes. Responsible for the overall system description, question bank administration, dice mechanics and turn management, and software system quality attributes. Sections: 2, 3.2.5, 3.2.8, 3.5.

Shira Marder - Board Configuration, Piece Movement, Game History, and Design Constraints. Responsible for board setup and configuration, piece movement and blocking rules, game history viewing, and architectural design constraints. Sections: 3.2.1, 3.2.2, 3.2.7, 3.4.

Stav Elkriyef - Introduction, Surprise Events, Question Handling, and Performance Requirements. Responsible for the document introduction, Surprise Square events, question selection and handling, and performance requirements. Sections: 1, 3.2.3, 3.2.4, 3.3.

Lana Admuz - External Interfaces, Game Data Persistence, Game Completion, and Other Requirements. Responsible for external interface requirements, completed-game data storage, game completion and ranking, executable packaging, and source-code documentation. Sections: 3.2.6, 3.2.9, 3.6, 3.1.